

03: Foundations

ISA 401: Business Intelligence & Data Visualization

2026-08-31

Slides | Class code

The recording is on Canvas. Timestamps before each paragraph are hh:mm:ss into the recording; drag the player there to hear the passage.

What we covered

The first half of the session was a Kahoot review of Week 01, which ran longer than Fadel intended because every question the room got wrong was stopped on and re-explained. The second half was the R setup we did not reach on Wednesday: an RStudio project inside your repository, an R Markdown file with a YAML header we built up option by option, and then the first R code, a single number and a named vector of seven crash counts.

- **Set up** an **RStudio project** inside your repository and work in **R Markdown**.
- **Create and inspect** objects and **vectors** with `=`, `c()`, `names()`, and `typeof()`.
- **Predict** how R handles **types**: one vector, one type, and a comparison returns a logical vector.

Before we start

00:00:03 A housekeeping note first: the notes on the Week 01 slides have been filled in, so if you go back to those decks you will see written out what was only said out loud in class. Wednesday's class was the motivation for R: it is a language you can read, and that lets you carry an analysis from one dataset to another far faster than you could in a spreadsheet, at least for Fadel. This week the focus moves from why to how.

00:01:20 The setup for the day: join the Zoom from the Week 0 module on Canvas so you can share your screen if you hit an error, and work from the computer that has your `business_intelligence` repository on it, because we add files to it today. If you are on a different machine, clone it first. RStudio is pinned to the taskbar on the lab machines; typing `RStudio` in the Windows search bar finds it. If you are on your own laptop, Assignment 0 already walked you through installing it.

Week 01 review on Kahoot

00:02:39 The plan is roughly five Kahoots, give or take, as a check on how the understanding is going.

Six questions today, evenly split between Class 01 and Class 02. When you join, use your first and last name so the bonus can be matched to you: if three people call themselves Andrew, that becomes work. Fadel played along on the projector but does not count, and said so twice.

Where business intelligence lives

00:05:49 The first question asked which two stages of the analytics journey business intelligence lives in, and the answer is pre-analytics and descriptive. The point of BI is to pull data, typically from several sources, and put it in front of people as an interactive dashboard so they can ask their own questions. The Job Scout example from Class 01 was both halves at once: scraping job ads, organizing them, and putting them in a readable table in a database is the pre-analytics part; the interactive charts on top, where you can ask for jobs in Ohio that want Tableau and Power BI, are the descriptive part. The data management piece is the work you did in 345. What this class adds is pulling data from sources outside your own organization.

00:07:24 Descriptive analytics itself splits in two: statistical summaries of the data, and visualizations people can understand. Predictive analytics is typically machine learning. Prescriptive got two buckets. One is experimental design, the sort of thing you would do in ISA 335: if Amazon moves its buy button to a different place and runs an A/B test, do more people buy? The other is the optimization side from ISA 321. You might have a good predictive model of how stocks will do over six months, but that does not mean you put all \$10,000 in one of them. You want to maximize return while minimizing risk, which means deciding how much of the \$10,000 goes into each stock.

00:08:55 BI is associated with dashboarding above all: generating insights from a dashboard in Tableau or Power BI, either retrospectively (how did we do last quarter) or in real time (in the Ford plant up in Michigan, what are we producing right now).

Three shapes of data

00:10:13 The second question was the JSON one, and the room did better on it. The three buckets, in the order the Job Scout example produces them: the wall of text describing a job is **unstructured**, because text and images are not structured by design. The skills that come back from the job-ads API arrive as **JSON**, and for a given job you might have ten skills. That is more organized than free text, but to put it in a data table you still have to decide *how*. Do you compress all the skills into one cell separated by commas or semicolons? Or do you build a long table with a job ID and a skill, so a job with five skills is five rows and a job with fifteen skills is fifteen rows? That decision is yours to make, and having to make it is what **semi-structured** means. **Structured** data is what you handled in 345: anything that already sits in a data table, the nice CSV you consume when you run a regression.

Data encoding, and why color is not encoding the counties

00:12:52 The third question showed the county-level map and asked how the counties are encoded. This one drew a pause, and Fadel used it to re-state two terms from Class 01.

00:13:13 The first is the **unit of analysis**: what does one row of data mean? In the crash dataset the unit of analysis was not a crash, it was a person involved in a crash. The second is **data**

encoding, which is a fancy way of asking how a variable in your file gets represented in a picture. On this map the counties are *drawn*: the x and y position on the map traces the border. Butler County sits where it does because it is in southwestern Ohio, next to Montgomery and Warren.

00:14:35 So why is color not the encoding for counties? Because color is not being used to tell Butler from Warren. The New York Times used color for the case count per capita: light orange means few cases, dark red or maroon means many. Nobody is saying Warren is orange and Cuyahoga is green as a way of telling them apart. Color encodes the case rate, not the county identity. Fadel checked twice that this had landed, noting the recording is there for it.

Commit, push, pull

00:15:43 The fourth question: you edit a file, save it, and commit it. Where does that file live now? The room got this one wrong enough that Fadel took it as an invitation to ask more questions in class.

00:16:21 Git and GitHub are different things, and Git operations happen on your computer. Ticking the checkbox in GitHub Desktop means you want to track a particular file. Clicking **commit** means you want a snapshot of that file as of 8:46:59 a.m. on August 31st. That is the equivalent of naming a version in a Google Doc history: this is the version I want to track, and if everything worked up to that point, this is the version I can come back to.

00:17:13 So for everyone who answered that the file is now on every computer where you are signed into GitHub Desktop: not yet. Committing does not put anything on the server. That is the **push** operation. And to have the file on a second computer you need two things: that you pushed it, and that you **pulled** the latest version of the repository onto that other machine. Pushing means it is on the cloud server at github.com, nothing more. If you want your laptop, or a gaming machine at home, to match your M drive, you open GitHub Desktop on that machine, go to the repository, and pull. Every other computer needs its own pull.

00:18:32 A note that sets the tone for the rest of the session: this is not a sprint. If the room needs something explained in more detail, it gets explained in more detail.

Which files can Git diff

00:18:58 The fifth question: for which file can Git show you exactly which lines changed?

00:19:21 Wednesday's slide split files into two colors, binary and not binary. Fadel could not recall which color went with which and said so. Binary files are things like a Word document, an image, a PDF. For those, Git can tell you the version changed, but not line by line what happened. For a Markdown file, a text file, a CSV (not an Excel file, a CSV), an R file, a Python file, anything flat, Git can tell you exactly what you changed. That is what we saw on Wednesday when a line was added to the skills list in the Markdown document: Git showed there was no line here before, and now there is one. Edit a line and it shows the old line and the new line side by side. With a .docx, a .png, or a PowerPoint, Git can save the prior version, but all it can tell you is that the new version differs, not how.

Undo versus revert

00:21:09 The last question: you pushed a commit with a mistake in it. What is the fix that keeps the history honest? Fadel asked the room to take the sentence apart first.

00:21:34 You pushed a commit, which is two things: you took a snapshot of the version containing the mistake, and you sent that snapshot up to GitHub. The mistake is already on the cloud. To keep the history honest you say: I made a mistake, that was version 2, and version 3 is going to be the state of things before the mistake. So the history reads as what you started with, then the mistake, then a new version that undoes it, with every step still visible. The term for that is **reverting**.

00:22:47 The other options do not work. There is no real undo in GitHub Desktop once you have pushed. Deleting the repository and starting over, which nobody picked, loses the history entirely. And pulling the latest repo from github.com just brings the mistake down to another computer, because the mistake is what is up there.

00:23:39 The winner of the Kahoot gets the 0.15 bonus on Assignment 02.

00:24:14 That review took about 25 minutes, more than intended. Fadel's view: if reviewing what we covered is what it takes for everyone to be on the same page going forward, that is a win, not a loss. The plan for the rest of the session: RStudio as the interface to R, a little syntax, and a lot of indexing, which is really basic, really fundamental, and the source of about half the errors you will get. Custom functions and `if/else` statements are nice-to-haves that may not happen at all; `for` loops will get their own treatment, maybe next week, maybe the week after.

Part 1: Your R project

How many of you have used R?

00:26:00 Before starting the demo, Fadel ran a quick Menti to set the pace: *Did you use R before?* All 27 responses said yes, but only 8 of them chose "Yes, comfortable with it" against 19 for "Yes, but I do not trust my skills". Nobody answered no. The plan from there was to take it one step at a time.

Making the repository an RStudio project

00:27:30 We will work exclusively in a project environment. The reason is that if we all have the folders set up the same way, the code Fadel writes will run on your computer without editing. On the projector this meant **File** → **New Project**, then **Existing Directory**, because the folder already exists from last class: on Fadel's M drive that is the `isa401a` folder, and on yours it should be `business_intelligence`. Browse to it, then **Create Project**.

00:28:32 What creating the project buys you is that every time you start an R session in it, the session starts *in that folder*. Paths are not `M:/Fadel/...` any more; everything is relative to the project root. More on what that means as the semester goes on.

00:29:10 A student asked whether that means you should have a separate project for each class that uses R. Yes, and it is a good recommendation. Somewhere between 10 and 20 percent of the

trouble people have reading files in R is that the file path is wrong: you are looking for `data.csv` and it is in Downloads, or on someone else's M drive, or wherever the professor kept it. That does not travel. If we all share the same folder structure, the code posted online runs as-is, as long as you are inside the project and you have downloaded any data file it needs.

R Markdown or R script?

00:30:00 A second student question, and then a question back to the room: R scripts or R Markdown? The room said Markdown, so Markdown it is, for this class and for the rest of the semester. **File** → **New File** → **R Markdown**, title `03 - R Basics`, and your own name in the author field.

00:30:48 With the file open, the four panes: the source file top left, the console below it where output gets shown and where you can try things you do not want in your record, the Environment pane where objects like a data frame show up, and the Files pane. The project name in the corner reads `isa401a`; in your case it should read `business_intelligence`.

00:31:27 Then a folder called `markdowns` inside the project, and the file saved into it as `03_r_basics.Rmd`.

00:32:12 Switching to GitHub Desktop to show where that landed: the repository shows a `.Rproj` file and the new R Markdown, and everything inside the R Markdown is green, meaning none of it existed before. (There is also a `.Rproj.user` folder, which we deal with shortly.)

RStudio settings

00:32:49 Part preference, part good practice. Fadel did this in **Tools** → **Project Options**, noting that the slide shows the same settings in Global Options, and that off the top of the head it was not clear which settings live in one and which in the other; they share a lot. Doing it per project is useful if another class has a faculty member with different preferences.

00:33:12 In **Project Options** → **General**, all three Workspace settings get set to no: *Restore .RData into workspace at startup*, *Save workspace to .RData on exit*, and *Always save history*. The reason: if you have a `DF` in your environment left over from a different session and a different dataset, you can quietly run your analysis on the wrong data. Fadel's position is that you should not have that.

00:33:58 In **Project Options** → **Code Editing**, tick **Use native pipe operator**, `|>`. On Windows, `Ctrl + Shift + M` then types the pipe for you (M as in Markdown, which is how Fadel remembers it). The pipe is a fancy way of saying the output of the first function becomes the first input to the next one.

00:34:34 A student following along could not reach Project Options: the entry was greyed out. That is the tell that you are not in a project. Create the project first, **New Project** → **Existing Directory**, browse to the folder you already have, **Create Project**, and Project Options becomes available. Fadel added, again, that the questions are welcome and this is not a race.

00:36:01 **Appearance** is optional, and was flagged as optional twice. Fadel uses the **Tomorrow Night Bright** editor theme. The only real advantage of matching it is that the colors on your

screen match the colors on the projector; the claim that a dark theme is easier on the eyes was offered as theory, not fact.

00:37:12 Then over to **Tools** → **Global Options** for the settings that are not available per project. Editor font size is a personal thing; Fadel likes it bigger. Under **Code** → **Display**, tick **Highlight R function calls** and **Use rainbow parentheses**. The first gives a function name its own color, distinct from the package name, instead of everything in the line being one color. The second gives each nesting level of parentheses a different color, so you can see at a glance whether you closed what you opened. Neither changes how the code runs; they just make your life easier.

00:38:51 Fadel then went looking for the R Markdown setting that sends chunk output to the console rather than inline, could not find it in that pane, checked Advanced, and moved on: “when we run it, I’ll figure it out.” It turns up later, at 01:06:37, under **Tools** → **Global Options** → **R Markdown: Show output preview in**, which offers **Window** and **Viewer Pane**.

The R Markdown cheat sheet

00:39:43 If you have used R Markdown and forgotten how something works, **Help** → **Cheat Sheets** → **R Markdown Cheat Sheet** downloads a PDF. Fadel noted there are two R Markdown entries in that menu and could not remember which was which. The one that opened lists the YAML options and shows what each format looks like, and it is two pages, as it always was.

The YAML header

00:40:37 The block between the two rows of three dashes, lines 1 to 6 of a new file, is the **YAML**, which stands for Yet Another Markup Language. Fadel hedged that the one-line gloss may not be fully accurate, but that as a mental model it is good enough: this is where you specify the metadata and how you want the file to look. Out of the box, clicking **Knit** produces an HTML file that lands next to the `.Rmd`, in `markdowns/`.

00:41:31 A small typing habit worth stealing: YAML is whitespace-sensitive in a way R is not, so instead of counting spaces, type the key and its colon first, then press Enter. RStudio indents the next line for you. Fadel deliberately wrote `html_document:` with the colon before pressing Enter to show the automatic indent.

00:42:14 From there the header gets built up one option at a time, knitting after each to show what changed. `toc: TRUE` adds a table of contents at the top of the document, which on the template file listed *R Markdown* and *Including Plots*.

00:42:52 `toc_float: TRUE` moves the table of contents to the left and makes it follow you as you scroll, which is what you want for the long documents you will end up writing. At this point Fadel also ticked **Knit on Save**, so that saving re-knits and the source and the output can be watched side by side.

00:43:26 `number_sections: TRUE` numbers the headings, so the two template sections become 1 and 2.

00:44:18 `code_folding: show` works only in HTML documents and does two things: a **Code** button appears at the top right of the knitted page with *Show All Code* and *Hide All Code*, and each chunk gets its own show/hide toggle.

00:44:48 `code_download: TRUE` is the one Fadel uses so that only one file has to be handed out. With it set, someone who has only the HTML can download the `.Rmd` from the page itself.

The header we ended up with is the one at the top of the class RMD:

```
#| eval: false
---
title: "03 - R Basics"
author: "Fadel M Fadel"
date: "`r Sys.Date()`"
output:
  html_document:
    toc: TRUE
    toc_float: TRUE
    number_sections: TRUE
    code_folding: show
    code_download: TRUE
---
```

Line 8

A table of contents at the top of the document.

Line 9

Moves it to the left margin and lets it float as you scroll.

Line 10

Numbers the sections.

Line 11

HTML only. Adds a **Code** button with show-all and hide-all, plus a toggle per chunk.

Line 12

Puts a download link for the `.Rmd` in the HTML, so one file is enough to hand out.

00:45:48 Most of these are documented in that R Markdown cheat sheet, in the YAML section, along with what each format looks like. The indentation trick is Fadel's way around the alignment rules: add the colon, press Enter, and let RStudio sort it out.

00:46:13 Last step before writing any R: everything in the file from the `R Markdown` heading down is template boilerplate that has nothing to do with this class, so delete it. Knit again and you get an empty page. Fadel noted this walkthrough will be much quicker in future classes; today it was worth showing as an option.

Debugging the YAML in the room

00:46:49 A student asked what the difference between the two code options is. `code_folding` is what gives you *Show All Code* and *Hide All Code*; `code_download` is what gives you the download link for the `.Rmd`. Both put a **Code** button on the page. If you add them and do not get the button, the problem is almost certainly the spacing.

00:47:57 Which is what it was. Two people shared their screens over Zoom, and the common failure both times was the YAML block rather than anything in R: the keys were not lined up with each other. The fix Fadel talked through is to stop counting spaces and rebuild the block: put the cursor at the end of `output:`, press Enter, type `html_document:`, press Enter again, and let RStudio place each following line. Each key ends up flush with its siblings without anyone counting. Once the indentation was consistent, knit succeeded.

Editorially: if you are debugging your own YAML later, the useful check is that every key at the same level starts in the same column, and that a key with children ends in a colon.

00:48:18 One practical request that came out of this: if you are joining the Zoom from your own computer, use the Zoom desktop app rather than Zoom in a browser tab. With the app, Fadel can ask for remote control and fix things directly instead of describing keystrokes.

Your first code chunk

00:53:33 A recommendation: never type the three backticks by hand. Use the insert-chunk button in the editor toolbar and pick R.

00:53:55 Reading a chunk header left to right: the first thing is the name of the *programming language*. R Markdown can run several languages in one document; Python needs a package called **reticulate**, but it runs. The second thing is the *name of the chunk*, and the recommendation is to give it a descriptive one so you know what you are looking at. Chunk names cannot contain spaces, so `cincy crashes` has to become `cincy_crashes`. After that come the chunk options, which we come back to.

The chunk that got named `cincy_crashes`, under the heading `# Crashes Per Day`, ended the session looking like this (its options are `results='hold'`, added at the very end of class):

```
#| eval: false
crashes_per_day = 30125 / 365
crashes_per_day

crashes_by_day = c(2020, 2241, 2327, 2285,
                  2380, 1866, 1664)
names(crashes_by_day) = c("Mon", "Tue", "Wed", "Thu",
                        "Fri", "Sat", "Sun")
crashes_by_day
```

Line 2

30,125 is the row count of the crash file, so this is crashes per day for 2025.

Line 3

Typing the name on its own line is how you print it.

When the chunk was first written it held only the two `crashes_per_day` lines; the vector arrives in Part 2.

00:54:24 Two ways to get output. **Knit** always saves the file first. **Knit on Save** is the other order: save, and it knits. Either way the knitted document has your code and your output together.

.gitignore, commit, push

00:54:50 The `.Rproj.user` folder that RStudio creates is not something anyone wants to see in your repository. In GitHub Desktop, right-click one of those changed files and choose **Ignore folder (add to .gitignore)**, then pick the top-level folder. That is not an error if you skip it, but those files are of no interest to anyone.

00:55:35 Ignoring the folder creates a new file called `.gitignore` in the repository with one line in it. On the projector this took the changes list from 19 files down to 4: the new `.gitignore`, the `.Rproj` file (which you probably do want to keep), and the `.Rmd` and `.html` in `markdowns/`.

00:56:15 Commit with a summary. Fadel typed one, misspelled it, retyped it, and committed. Then, deliberately, checked GitHub before pushing: the repository page still showed nothing new, last commit five days ago. Only after clicking **Push origin** and refreshing did the `markdowns` folder appear on github.com.

00:57:26 Fadel also spent a moment looking for a way to suppress an inline preview element in the editor and did not find it: “there used to be a way, maybe in this version of RStudio there’s not.” Not important, but it is why the screen looked slightly different from previous years.

00:58:10 Two ways to run code in a chunk: `Ctrl + Enter` runs the current line, or you can run the whole chunk at once. Fadel writes one line at a time and runs it, to be sure it works before moving on.

00:58:35 The ritual for every class from here: open the project, work in an R Markdown (the room chose Markdown, so Markdown it is), and before you leave, knit, commit, and push. That is how everything stays in sync.

Part 2: Objects and vectors

Three assignment operators

00:59:05 R is a weird programming language in one specific way: you can assign a value to an object in three different ways.

```
#| eval: false
crashes_per_day = 30125 / 365
crashes_per_day <- 30125 / 365
30125 / 365 -> crashes_per_day
```

Line 2

=, which is what you have most likely used in your other classes and what we use here.

Line 3

<-, literally an arrow: take this value and assign it back into the name. This is what you get if you ask ChatGPT for R code or find code online, and it was the default way people wrote R, Fadel guessed, ten or fifteen years ago.

Line 4

->, pointing the other way. Uncommon, but Fadel does use it at the end of a series of piped steps, where saying “and the output goes into this object” reads well.

01:00:13 All three do the same thing. Fadel did not know whether other languages offer this and said so; the point is only that R has three and that this is unusual.

Object not found

01:00:13 R’s errors are awful. They are not very descriptive. The most common one you will meet is `object ... not found`, and Fadel simulated it live by deleting `crashes_per_day` from the environment with `rm()` and then pressing `Ctrl + Enter` on the line that prints it. R has no idea where the object is and cannot help you.

01:01:11 That single message has three causes, and Fadel’s ranking of them: you capitalized something that should not be capitalized, you have some other typo in the name, or you simply never ran the line. The easiest way to check is to look at whatever you wrote and then look at the Environment pane. If it is not there, that is why you are getting the error.

R is a vector language

01:01:38 The interesting thing about R is that it is a vector language: you can run a function on a whole vector and it works, element by element, without you writing a loop. The seven numbers used here are the counts of crashes on each day of the week in 2025, the same seven bars from Wednesday’s chart.

```
#| eval: false
crashes_by_day = c(2020, 2241, 2327, 2285,
                  2380, 1866, 1664)
names(crashes_by_day) = c("Mon", "Tue", "Wed", "Thu",
                          "Fri", "Sat", "Sun")

crashes_by_day
crashes_by_day > 2000
sum(crashes_by_day > 2000)
mean(crashes_by_day)
```

Line 2

`c()` puts the numbers together into a vector of seven elements. Fadel thinks of it as concatenating, while noting that whether that is technically what is happening is another matter.

Line 4

`names()` attaches a name to each element, making it a named vector. Printing it now shows Mon at 2020, Tue at 2241, and so on.

Line 7

A greater-than sign in R is a logical operation: it asks, for each element, true or false. One comparison, seven answers. On the slide, Saturday and Sunday come back `FALSE` and the other five `TRUE`.

Line 8

`sum()` of `TRUE`s and `FALSE`s counts the `TRUE`s, because `TRUE` is 1 and `FALSE` is 0. The slide shows 5.

Line 9

`mean()` works on the vector the same way, and the slide shows 2111.857.

01:03:36 No `for` loop anywhere. Fadel's contrast: in Python the default way to do this would be to go through the list, typically with a list comprehension. In R you operate on the vector itself.

Types

01:04:22 R has six data types by default, four of which are common enough that this class will highlight them. **Logical** is just `TRUE` or `FALSE`. **Integer** is a whole number, and to make something an integer in R you give it an uppercase `L`.

01:04:48 The demonstration was a bit of a fumble over which object was called what, which is worth watching for its own sake: Fadel had to go back, check the name in the earlier chunk, and copy it rather than retype it. Under a new heading, `## Type of `crashes_per_day`` (the backtick is the key next to the 1, and it makes the name render as code when you knit), the second chunk of the class RMD is:

```
#| eval: false
typeof(crashes_by_day)

crashes_by_day_int = c(2020L, 2241L, 2327L, 2285L,
                      2380L, 1866L, 1664L)
names(crashes_by_day_int) = c("Mon", "Tue", "Wed", "Thu",
                             "Fri", "Sat", "Sun")
crashes_by_day_int |> typeof()
```

Line 2

In the console this returned `"double"`.

Line 4

The same seven numbers, copied from the chunk above rather than retyped, with an uppercase `L` after each one to make them integers. Fadel said out loud not to type these by hand.

Line 6

Copying the block over left this line still naming `crashes_by_day`; Fadel spotted it and changed it to `crashes_by_day_int`, adding that it did not really matter here, because the only thing being checked was the type.

Line 8

The native pipe: take the thing on the left and make it the first input to the function on the right. `Ctrl + Shift + M` types the pipe. This returned `"integer"`.

01:09:31 The pipe is unnecessary here and Fadel said so; it was shown only to show it. The substance is the contrast between the two: the first vector had no decimal points anywhere and R still stored it as a double, because a double is what allows decimals. If you want something to be explicitly an integer, you have to tell R.

`results='hold'`

01:07:04 Along the way, the `results='hold'` chunk option, which Fadel uses in the class notes. Instead of printing the first result, then showing more code, then printing the next result, it holds the outputs and shows them together. Looking at the held output is what surfaced a mistake: the chunk was printing `crashes_per_day` where it should have printed `crashes_by_day`. Fixed, the knitted chunk shows two things, a single value and the named vector.

01:06:37 This is also where the setting Fadel had been hunting for earlier turned up. **Tools** → **Global Options** → **R Markdown** → **Show output preview in** takes either **Window** or **Viewer Pane**: one opens the knitted file in its own window, the other shows it in the pane on the right. It is up to you.

Four types, and the trap

01:10:45 Back to the slides for a visual view of the four types. **Logical** takes two values. **Factor** is the one you have met in regression: if you had days of the week as a predictor, you made them a factor. A factor is another way of saying categorical variable, and it differs from a character variable in that it is limited to specific values. If the days of the week are Sunday, Monday, Tuesday, Wednesday, then those are the levels, and you cannot have a day of the week called something else. That is the point of it: you specify the levels the variable is allowed to take. Fadel's analogy was a select filter in a dashboard, which limits people's answers to a fixed list.

01:12:07 **Integer** and **double** are the numeric types. Fadel misspoke and immediately corrected: double allows decimal points, integer does not.

01:12:33 To check the type of something, `typeof()`, and you can hand it a single value or a whole vector.

The self-assessment

01:12:33 Two minutes on the clock to connect each of six columns from the Cincinnati crash file to its type: `instanceid`, `crashdate`, `dayofweek`, `age`, `latitude_x`, `injuries`. The seven sample rows on screen show what the values look like.

01:13:03 A caveat given before the timer started, and it matters: the note at the bottom says the data was read with `read_csv()`. Some of these columns could be understood differently depending on how you read the file. A column full of dashes and colons might well come back as character; here it does not.

01:15:10 Working through them. `instanceid` looks like words, like text, which is the usual indication of character.

01:15:32 `crashdate` depends on how you read the file, and Fadel put the question to the room: when you read CSV files in R, what function do you use? A student answered `read.csv`. With `read.csv` this would come back as text and that would be fine. But the **readr** package has a function called `read_csv`, which guesses types better: it recognizes this as a date-time, which makes it a double. It is numeric underneath.

01:16:13 `dayofweek` is text, so character. `age` is the one that catches people. It should be numeric, but the column contains some non-numeric values, and that makes the whole thing character.

01:16:39 That is the rule worth carrying out of today, and it is not only an R rule: a vector can hold only one type. Mix types in it and R generalizes to the most general type that can hold everything. Integers and doubles together, say `401L` and `44.7`, generalize to double, because a double can represent both. Numbers and text together generalize to character.

01:17:14 So in the table on screen, `age` shows `47` in one row and a letter code in another, and that makes the column character. `latitude_x` obviously has a decimal point, so it is a double, and `injuries` is obviously text.

01:17:43 The Answer tab on the slide has the code that produces the six answers, and the two gotchas written out: `crashdate` is a double because readr stores a date-time as a number with a label on top, and `age` is character because of the letter codes in it. Cleaning `age` is a Class 07 job. Fadel invited notes from the discussion to go into the editable box on that slide, which is where they belong if you were taking any.

Wrapping up

01:18:02 With two minutes left, Fadel chose not to push into the next topic and instead closed the Git loop. The changed files show in orange this time rather than green, because they already existed: a block of changed lines in the HTML, starting around line 1600, and in the `.Rmd` the chunk gained `results='hold'` and a few added lines. Commit summary `end of class03`, then push.

01:19:17 One last Git point, made with the commit sitting locally and not yet pushed. Because it has not been pushed, right-clicking it offers **undo commit**, and undo does not delete your work: it just puts the changes back so you can add whatever you forgot. If you had already pushed, that option is gone and you revert instead, which adds the extra step to the history. This is the same undo-versus-revert distinction the Kahoot asked about, seen from the other side.

01:19:45 Next class continues with R: data frames and subsetting. The two assignments were pushed by a day; you can do them now, because nothing in them depends on where we stopped

and there were no R questions in them. The first fifteen minutes of Wednesday go to a former student now at Deloitte, talking about recruiting.

01:20:29 The takeaway Fadel asked for: questions in class are always welcome, so stop the lecture and ask. And if you are chatting on Zoom, use the app rather than the browser, so screen shares can be driven from the podium.

Questions from class

00:29:10 A student asked whether you should set up a separate project directory for each class that uses R. Yes. A project per class, and the benefit is that everyone's paths then agree, so posted code runs without editing.

00:30:00 Asked in return, and answered by the room: R scripts or R Markdown? Markdown, and that is what the course uses from here.

00:34:34 A student pointed out that Project Options was greyed out. That is what it looks like when you are not inside a project: create the project first, **New Project → Existing Directory**, and the entry becomes available.

00:46:49 A student asked what `code_folding` and `code_download` each do, since both put a Code button on the page. `code_folding` gives you show-all and hide-all plus a per-chunk toggle; `code_download` gives you the link to download the `.Rmd`.

01:15:32 Asked of the room: which function do you use to read a CSV in R? Someone answered `read.csv`. That works, and Fadel used it to introduce `readr::read_csv()`, which guesses column types better and is why `crashdate` comes back as a date-time rather than text.

Loose ends